

MIND: MRI-based Intelligent Neural Detection

A deep learning system for brain tumor classification from MRI

Huynh Mai Linh Nguyen

2026

Abstract

We present **MIND** (MRI-based Intelligent Neural Detection), a compact research prototype for four-class brain MRI image classification (Glioma, Meningioma, No Tumor, Pituitary Tumor). EfficientNet-B0 is fine-tuned from ImageNet initialization with standard preprocessing, light augmentation, and a stratified 85/15 train/validation split on the Kaggle Brain Tumor MRI dataset. In our final CPU run (15 epochs; 4,760 training and 840 validation images), peak validation accuracy reached **99.52%** at epoch 8 (final epoch: **99.05%**). A Streamlit interface enables interactive upload-based inference with per-class probabilities. The contribution is **educational and exploratory**; outputs are not intended for clinical decision-making.

Keywords: brain MRI; brain tumor classification; deep learning; EfficientNet-B0; transfer learning; Streamlit.

Medical Disclaimer

This software and report are provided for **educational and research purposes only**. The system is **not** designed to diagnose, treat, or prevent disease, and it does **not** replace the judgment of a licensed physician or other qualified healthcare professional. All medical decisions should be made by appropriately credentialed clinical staff.

1 Introduction

Brain tumors are a serious class of pathology that benefits from early and accurate assessment. Magnetic resonance imaging (MRI) is a key modality for evaluating intracranial lesions [1, 3]. Machine-assisted classification of tumor type from MRI may support radiologists and trainees in research and teaching settings [2].

MIND implements an end-to-end deep learning pipeline: collect the Kaggle dataset, train EfficientNet-B0 in PyTorch [9], save the checkpoint `weights/best_model.pth`, and deploy a Streamlit front end (`app.py`) for CPU inference [11].

2 Background

2.1 Convolutional neural networks

Convolutional neural networks (CNNs) learn spatial filters through convolution and pooling layers, which is well suited to medical imaging [6]. Deeper layers often encode features from edges and textures to lesion morphology.

2.2 Transfer learning

Rather than training from scratch on a relatively small MRI set, we initialize the backbone with weights pretrained on ImageNet [4] and fine-tune the classifier head for the four brain-tumor classes. This follows transfer-learning practice [8, 14, 12] and typically converges faster.

2.3 EfficientNet-B0

EfficientNet [13] balances depth, width, and input resolution via compound scaling. The B0 variant is lightweight and practical for CPU deployment and Streamlit demos. In MIND we use ImageNet weights (IMAGENET1K_V1) [4] and replace the final `Linear` layer with four output units.

3 Data

3.1 Source

The project uses the **Brain Tumor MRI** dataset on Kaggle [7]. Download and extract it under `data/` using the layout below.

3.2 Directory layout

Table 1: Data directory layout (Training / Testing)

Path	Description
<code>data/Training/glioma/</code>	Glioma images
<code>data/Training/meningioma/</code>	Meningioma images
<code>data/Training/notumor/</code>	No-tumor images (or folder <code>no_tumor</code>)
<code>data/Training/pituitary/</code>	Pituitary-region tumor images
<code>data/Testing/...</code>	Optional; <code>train.py</code> trains on Training/ only

3.3 Four-way classification

Table 2: Class index, display name, and folder mapping (FOLDER_TO_IDX in `train.py`)

Index	CLASS_NAMES	Kaggle folder	Brief description
0	Glioma	glioma	Glial tumor
1	Meningioma	meningioma	Meningeal tumor
2	No Tumor	notumor, no_tumor	No tumor detected
3	Pituitary Tumor	pituitary	Pituitary-region tumor

Training recursively collects `.jpg`, `.jpeg`, `.png`, `.bmp`, `.tif`, `.tiff`, and `.webp` files in each class folder (see `IMAGE_EXTENSIONS` in `train.py`).

4 Methods

4.1 Preprocessing

- Resize images to 224×224 pixels (EfficientNet-B0 input size).
- ImageNet normalization: mean = $[0.485, 0.456, 0.406]$, std = $[0.229, 0.224, 0.225]$.
- Convert grayscale or other RGB modes to RGB before applying transforms (see `src/inference.py`).

4.2 Data augmentation (training)

On the training split: `RandomHorizontalFlip`, `RandomRotation` (15°). The validation split uses only resize and normalization.

4.3 Data split

`sklearn.model_selection.train_test_split` with stratify on labels:

- Training: 85%
- Validation: 15%
- `random_state = 42` for reproducibility.

4.4 Loss and optimization

Cross-entropy loss (`nn.CrossEntropyLoss`) and the Adam optimizer [5]. The best checkpoint (by validation accuracy) is saved to `weights/best_model.pth`.

5 Model architecture

The model is defined in `src/model.py`:

1. Load `efficientnet_b0` with ImageNet weights.
2. Replace `model.classifier[1]` with `nn.Linear(in_features, 4)`.

3. Inference: softmax on logits; predicted class is the argmax probability (see `predict()` in `src/inference.py`).

Table 3: Output-side architecture summary

Component	Value
Backbone	EfficientNet-B0 (ImageNet pretrained)
Input	Tensor $1 \times 3 \times 224 \times 224$
Output	Four logits $\rightarrow$ four brain-tumor classes

6 Experiments

6.1 Hyperparameters

Table 4: Default training hyperparameters (`train.py`)

Parameter	Value
Epochs	15
Batch size	32
Optimizer	Adam
Learning rate	1×10^{-4}
Validation split	0.15 (15%)
Seed	42
Device	CUDA if available, otherwise CPU

Example training command:

```
python train.py --data_dir data --epochs 15
```

6.2 Evaluation notes

The final run reports best validation accuracy from the console log line **New best val acc.** Confusion-matrix and training-curve visualizations are discussed in text in this submission.

7 Results

The values below summarize the final training run on CPU.

7.1 Accuracy

Best validation accuracy (15% hold-out):

Best validation accuracy: 99.52% (epoch 8)

Run setting summary: CPU training with 4,760 training images and 840 validation images for 15 epochs. Final-epoch validation accuracy (epoch 15): **99.05%**. The best checkpoint was saved to `weights/best_model.pth`.

The model converged rapidly during the first epochs and reached its highest validation accuracy at epoch 8. After this point, training accuracy continued to improve while validation accuracy plateaued and then declined slightly by epoch 15, which is consistent with a mild late-epoch overfitting signal. Given this behavior, selecting the checkpoint with the highest validation accuracy provides a conservative and standard criterion for model selection, as it prioritizes generalization on unseen validation data over continued optimization on the training split.

Additional metrics (per-class precision, recall, F1) were not computed in this final submission run.

7.2 Confusion matrix

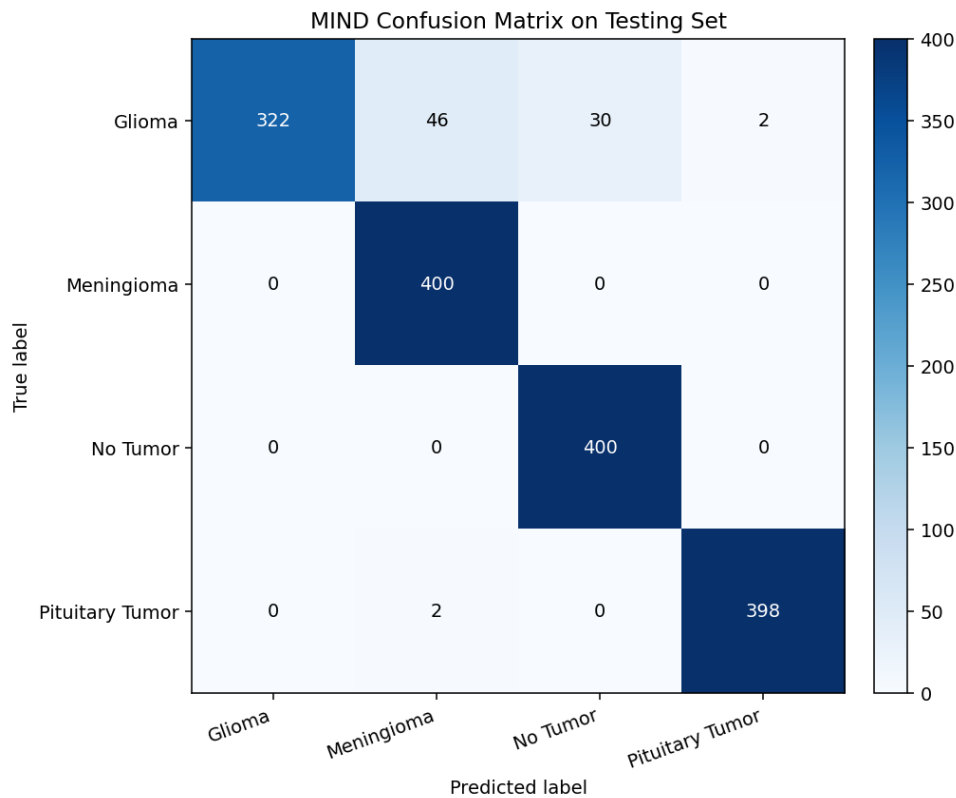


Figure 1: Confusion matrix on `data/Testing` using the best checkpoint (CPU inference).

Figure 1 summarizes class-wise predictions on the testing split with the same class-index mapping used during training and inference.

7.3 Training curves

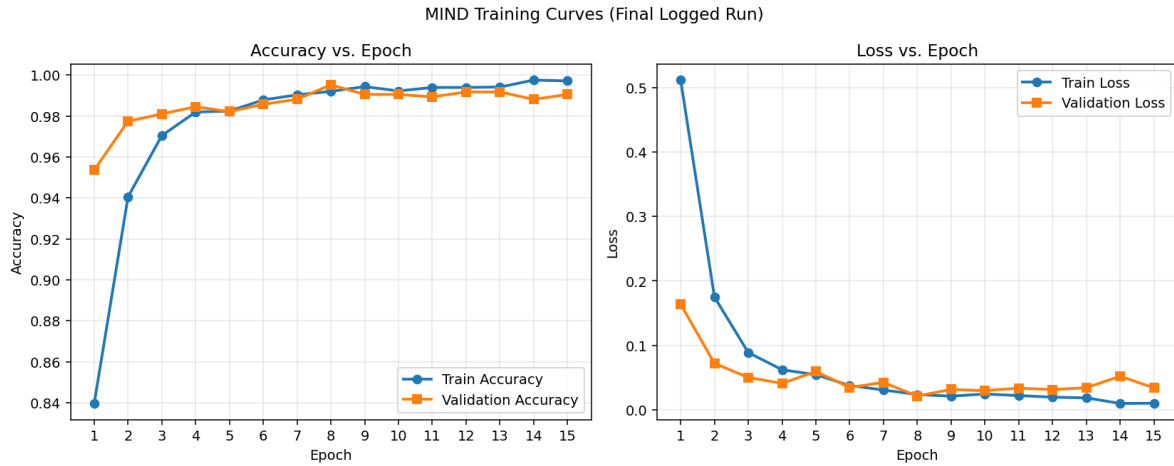


Figure 2: Training and validation accuracy/loss curves for the final 15-epoch run.

Figure 2 shows rapid early convergence, peak validation performance at epoch 8, and a modest late-epoch dip that suggests mild overfitting beyond the best checkpoint.

8 Streamlit web application

The `app.py` demo provides [11]:

1. **Page config:** MIND title, wide layout, sidebar hidden.
2. **Weight check:** if `weights/best_model.pth` is missing, show an error and disable inference.
3. **Upload:** `st.file_uploader` (JPG, JPEG, PNG).
4. **Display:** left column — uploaded image; right column — predicted class, confidence, progress bar, bar chart of class probabilities, and percentage table (`CLASS_NAMES` order matches `src/inference.py`).
5. **Model cache:** `@st.cache_resource` loads `load_model` once.
6. **Inference:** `predict()` in `src/inference.py` (preprocess $\rightarrow$ forward $\rightarrow$ softmax).
7. **Footer:** author name, copyright 2026, and medical disclaimer.

Run the app:

```
streamlit run app.py
```

9 Limitations and future work

- Labels and distribution come from the Kaggle corpus and may not match real-world clinical practice.
- Grad-CAM integration in `src/gradcam.py` for spatial explanations [10] is not wired into the UI yet.
- The pipeline uses 2D slices only; multi-sequence 3D MRI is not modeled.
- CPU-focused deployment suits a demo; latency and memory at scale need separate evaluation.

Future work: complete Grad-CAM in Streamlit; evaluate on `data/Testing/`; probability calibration; prospective validation with clinicians in controlled research settings.

10 Conclusion

MIND demonstrates a brain-tumor MRI classification pipeline with EfficientNet-B0, from Kaggle data preparation and stratified validation training to a Streamlit learning demo. The final run reached **99.52%** best validation accuracy (epoch 8) and **99.05%** final validation accuracy (epoch 15). The system is for education and research only and does not replace clinical diagnosis.

References

- [1] Sabaa Ahmed Yahya Al-Galal, Imad Fakhri Taha Alshaikhli, and Mohammed Mosa Abdulrazzaq. Mri brain tumor medical images analysis using deep learning techniques: A systematic review. *Health and Technology*, 11(2):267–282, 2021.
- [2] Milica M. Badza and Milan C. Barjaktarovic. Classification of brain tumors from mri images using a convolutional neural network. *Applied Sciences*, 10(6):1999, 2020.
- [3] Endalamaw Biratu, Friedhelm Schwenker, Yaregal Assaye Ayano, and Tewodros Geberesslassie Debelee. A survey of brain tumor segmentation and classification algorithms. *Journal of Imaging*, 7(9):179, 2021.
- [4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [5] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 2015.
- [6] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [7] Masoud Nickparvar. Brain tumor mri dataset. Kaggle, 2023. Accessed May 20, 2026.
- [8] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.

- [9] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8026–8037, 2019.
- [10] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 618–626, 2017.
- [11] Streamlit Inc. Streamlit documentation. <https://docs.streamlit.io/>, 2026. Accessed May 20, 2026.
- [12] Nima Tajbakhsh, Jae Y. Shin, Suryakanth R. Gurudu, Robert T. Hurst, Christopher B. Kendall, Michael B. Gotway, and Jianming Liang. Convolutional neural networks for medical image analysis: Full training or fine tuning? *IEEE Transactions on Medical Imaging*, 35(5):1299–1312, 2016.
- [13] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114, 2019.
- [14] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems 27*, pages 3320–3328, 2014.